

Secret Management Solutions

Comparative Evaluation & Proof of Concept

HashiCorp Vault | OpenBao | Vaultwarden

Prepared for:	DevOps / Infrastructure Team
Scope:	SMS Platform Secrets (Grafana, dlrsender, Erlang Cluster)
Document type:	Technical POC Report
Date:	08 July 2026
Status:	Final

Table of Contents

1.	Introduction & Background
2.	Objective and Scope of the POC
3.	Solutions Shortlisted for Evaluation
4.	Comparison Matrix (All Evaluated Parameters)
5.	POC Methodology
6.	Phase 1 — API-Based Secret Retrieval Testing
6.1	HashiCorp Vault — curl / API test
6.2	OpenBao — curl / API test
6.3	Vaultwarden — API limitations
6.4	Phase 1 Summary
7.	Phase 2 — CI/CD Integration Testing (Jenkins)
7.1	HashiCorp Vault — Jenkins Pipeline
7.2	OpenBao — Jenkins Pipeline
7.3	Vaultwarden — CI/CD Suitability
7.4	Phase 2 Summary
8.	Evaluation Against Requested Parameters
9.	Pros and Cons per Solution
10.	Scorecard
11.	Recommendation / Final Verdict
12.	Conclusion

1. Introduction & Background

The SMS department currently operates a Docker Compose based stack (`docker-compose-sms-local-poc-openbao.yml`) that was originally created to bring up several containers for local testing, including the department's monitoring dashboard (Grafana) and application services such as `dlrsender` and an Erlang-based cluster (RabbitMQ/messaging layer). This compose file was already in routine use to spin up these containers.

During review of this stack it was observed that a number of sensitive values were stored as plain environment variables directly inside the compose file and associated `.env` configuration, namely:

- **Grafana admin credentials** — `GF_SECURITY_ADMIN_USER` and `GF_SECURITY_ADMIN_PASSWORD` , used to protect the monitoring dashboard.
- **dlrsender repository/service secrets** — credentials associated with the dlrsender application container.
- **Erlang cluster cookie** — the `NODE_COOKIE` value used for inter-node authentication within the Erlang/RabbitMQ cluster. Anyone possessing this value can join or interact with the cluster.

Because these values were stored in plaintext inside version-controlled/compose configuration, this was flagged as a **security concern**. The requirement was raised to move all such secrets into a dedicated secret management solution rather than leaving them embedded in compose files, so that credentials can be centrally stored, access-controlled, audited, and injected into containers and pipelines at runtime instead of being hard-coded.

This report documents the comparison and hands-on Proof of Concept (POC) carried out to identify the most suitable secret management solution for this environment, using the same compose stack (repurposed to also stand up the candidate secret managers) as the basis for testing.

Note on the environment used: The same Docker Compose file that is normally used to bring up the SMS local POC stack (Grafana, dlrsender, Erlang cluster, and related services) was reused/extended to additionally bring up the secret management containers (HashiCorp Vault, OpenBao) being evaluated in this POC, so that the exercise closely reflects the real deployment topology.

2. Objective and Scope of the POC

The objective of this exercise is to evaluate secret management tools from a DevOps and infrastructure perspective and to identify the solution that best fits the department's requirements, covering:

- Installation and setup complexity
- Secret storage capabilities
- Access control and authentication mechanisms
- Audit logging and security features
- API and automation support
- Integration with CI/CD tools (Jenkins, GitLab, etc.)
- Kubernetes and container platform support
- High availability and backup options
- Ease of administration
- Licensing and cost considerations
- Community support and documentation

As part of the POC, each shortlisted solution was deployed and configured, sample secrets (Grafana credentials and the Erlang `NODE_COOKIE`) were stored and retrieved, API-based secret retrieval was tested wherever technically possible, and the setup process, observations, advantages, limitations, and recommendations were documented with supporting screenshots.

3. Solutions Shortlisted for Evaluation

Three solutions were shortlisted for a hands-on POC based on their relevance to an open-source, self-hosted DevOps environment:

Solution	Category	Why Shortlisted
HashiCorp Vault	Enterprise-grade secrets engine (OSS core)	Industry-standard secret manager with mature KV storage, dynamic secrets, ACL policies, audit devices, and a first-class HTTP API — widely used in DevOps/CI-CD pipelines.
OpenBao	Open-source Vault fork (Linux Foundation)	Community-governed, fully open-source fork of Vault (created after HashiCorp's BUSL relicensing), API- and CLI-compatible with Vault, free of licensing ambiguity.
Vaultwarden	Bitwarden-compatible password manager	Lightweight, self-hosted password vault; evaluated to see whether a "team password manager" style tool can double up as an infrastructure/DevOps secret store.

Other enterprise/managed options were considered during desk research but were not deployed for hands-on testing in this round, since they either require a paid/managed subscription or a cloud account, or fall outside the "open-source, self-hostable" requirement for this environment:

Tool	Type	Reason not selected for hands-on POC
Infisical	Open-source secret manager	Newer project; considered a good future candidate but not included in this round of hands-on testing.
AWS Secrets Manager	Managed cloud service	Requires an AWS account and incurs a per-secret/API-call cost; ties secret storage to a single cloud provider.
Azure Key Vault	Managed cloud service	Same constraints as above, but for Azure; not aligned with the on-prem/self-hosted nature of the current stack.
CyberArk Conjur	Enterprise secrets platform	Strong enterprise feature set but heavier footprint and commercial licensing for full functionality.

4. Comparison Matrix (All Evaluated Parameters)

Parameter	HashiCorp Vault	OpenBao	Vaultwarden
Installation & setup	Moderate — single binary/Docker image, needs init/unseal, storage backend choice	Moderate — identical workflow to Vault (fork), Docker image, init/unseal	Easy — single Docker container, minimal config to get a working vault
Secret storage model	KV v1/v2, dynamic secrets (DB, cloud, PKI, SSH), leasing/TTL	Same as Vault — KV v1/v2, dynamic secrets engines, leasing/TTL	Encrypted "items" (logins, notes, cards) designed for end-user credentials, not dynamic/leased secrets
Access control & auth	Policies (HCL), tokens, AppRole, LDAP, OIDC, Kubernetes auth, MFA	Same auth methods as Vault (compatible engine)	User/organization based access via Bitwarden accounts & collections; no machine-oriented auth methods (no AppRole/K8s auth)
Audit logging	Built-in audit devices (file, syslog, socket) — full request/response audit trail	Same audit device model as Vault	Basic event logging in the web UI (item created/updated); no dedicated audit-device/API for automation
API & automation support	Full REST HTTP API, official CLI, SDKs for Go/Python/etc., Terraform provider	Full REST HTTP API (Vault-compatible), CLI (<code>bao</code>), same SDKs largely work	Account-oriented API for the Bitwarden clients; secret retrieval as a "vault item" requires the authenticated Bitwarden/CLI login flow rather than a single token-based REST call
CI/CD integration (Jenkins/GitLab)	Native Jenkins & GitLab plugins, Vault Agent, sidecar injectors	Works with the same plugins/Agent pattern as Vault (API-compatible)	No native CI/CD plugin or pipeline-friendly machine-auth method
Kubernetes / container support	Official Helm chart, Vault Agent Injector, CSI provider	Community Helm chart, CSI provider (project is younger but active)	Runs fine as a container, but no K8s-native secret injection tooling
High availability & backup	HA with Consul/Integrated Storage (Raft), snapshot/restore	HA with Integrated Storage (Raft), snapshot/restore (same as Vault)	Single-instance by design; backup = copy of the SQLite/Postgres DB & attachment files
Ease of administration	Powerful but steeper learning curve; feature-rich Web UI	Same UI/CLI as Vault, so administration effort is comparable	Very simple, familiar Bitwarden-style Web UI — easiest to administer for end users
Licensing & cost	Community edition BUSL-1.1 (source-available, use restrictions apply); Enterprise is paid	Fully open source — MPL-2.0, governed by Linux Foundation, no usage restrictions	Open source (AGPL-3.0), completely free to self-host
Community support & docs	Very large community, extensive official docs, large ecosystem	Growing rapidly post-fork, backed by Linux Foundation, documentation closely mirrors Vault's	Active community around Bitwarden/Vaultwarden, good docs for password-manager use cases

5. POC Methodology

The POC was executed in two phases to specifically validate the two capabilities that matter most for a DevOps use case — programmatic (API-based) secret retrieval, and integration with the existing CI/CD tool (Jenkins):

1. **Phase 1 — API-Based Testing:** Each solution was deployed via Docker (re-using/extending the existing SMS compose file), unsealed/initialized, and a sample secret set (Grafana admin credentials, Erlang `NODE_COOKIE`) was written and then retrieved directly using `curl` against the HTTP API, from a Linux terminal.
2. **Phase 2 — CI/CD Integration Testing:** A Jenkins pipeline job was created for each candidate to validate that secrets could be pulled into a build/deploy stage the same way they would be consumed by an automated pipeline in production.

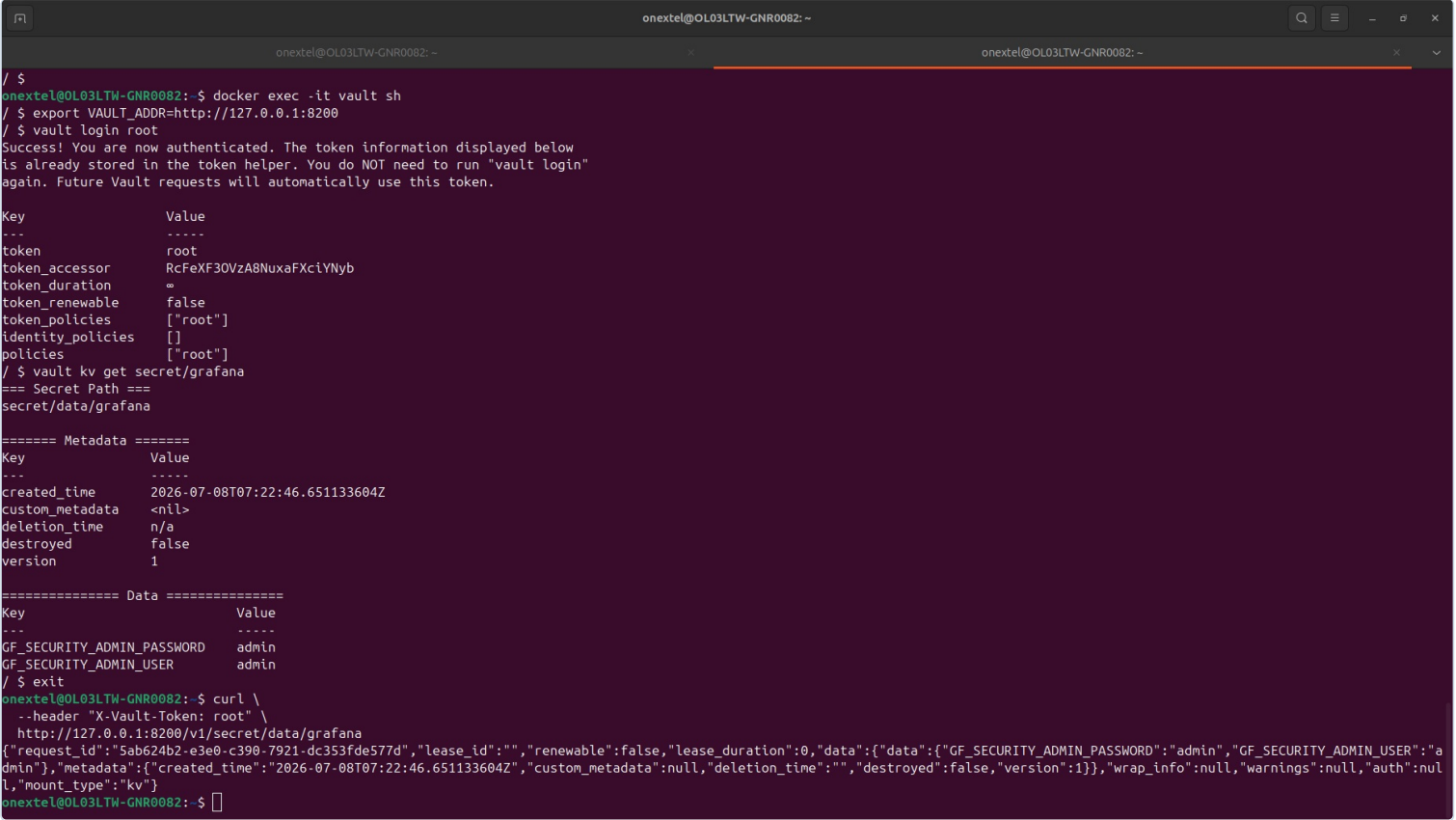
All screenshots included below were captured directly from the test environment during this POC.

6. Phase 1 — API-Based Secret Retrieval Testing

This phase validates whether each tool's HTTP API can be used the way an automation script or pipeline step would use it: authenticate once, then issue a single `curl` request to fetch a secret value as structured (JSON) data.

6.1 HashiCorp Vault — curl / API Test

Vault was started as a container from the compose stack, initialized with Shamir seal (single share/threshold for this POC), and logged into using the root token. The Grafana secret was written to the `secret/` KV engine and read back first via the `vault kv get` CLI (to confirm the data), and then via a raw `curl` call against `/v1/secret/data/grafana` using an `X-Vault-Token` header — returning the secret as a JSON payload, exactly as a script or Jenkins stage would consume it.



```
onextel@OL03LTW-GNR0082: ~  
/ $  
onextel@OL03LTW-GNR0082:~$ docker exec -it vault sh  
/ $ export VAULT_ADDR=http://127.0.0.1:8200  
/ $ vault login root  
Success! You are now authenticated. The token information displayed below  
is already stored in the token helper. You do NOT need to run "vault login"  
again. Future Vault requests will automatically use this token.  
  
Key          Value  
---          -  
token        root  
token_accessor RcFeXF30VzA8NuxaFXciYnyb  
token_duration 1  
token_renewable false  
token_policies ["root"]  
identity_policies []  
policies      ["root"]  
/ $ vault kv get secret/grafana  
=== Secret Path ===  
secret/data/grafana  
  
===== Metadata =====  
Key          Value  
---          -  
created_time 2026-07-08T07:22:46.651133604Z  
custom_metadata <nil>  
deletion_time n/a  
destroyed     false  
version       1  
  
===== Data =====  
Key          Value  
---          -  
GF_SECURITY_ADMIN_PASSWORD admin  
GF_SECURITY_ADMIN_USER   admin  
/ $ exit  
onextel@OL03LTW-GNR0082:~$ curl \  
  --header "X-Vault-Token: root" \  
  http://127.0.0.1:8200/v1/secret/data/grafana  
{  
  "request_id": "5ab624b2-e3e0-c390-7921-dc353fde577d",  
  "lease_id": "",  
  "renewable": false,  
  "lease_duration": 0,  
  "data": {  
    "data": {  
      "GF_SECURITY_ADMIN_PASSWORD": "admin",  
      "GF_SECURITY_ADMIN_USER": "admin"  
    },  
    "metadata": {  
      "created_time": "2026-07-08T07:22:46.651133604Z",  
      "custom_metadata": null,  
      "deletion_time": "",  
      "destroyed": false,  
      "version": 1  
    },  
    "wrap_info": null,  
    "warnings": null,  
    "auth": null,  
    "mount_type": "kv"  
  }  
}
```

Figure 1 — Linux terminal: `docker exec` into the Vault container, `vault login root`, `vault kv get secret/grafana`, followed by a `curl` request with an `X-Vault-Token` header against `http://127.0.0.1:8200/v1/secret/data/grafana`, returning the Grafana admin credentials as JSON.

Supporting evidence — the same secret and the Erlang cluster secret as they appear in Vault's own Web UI, and the full list of secrets migrated into Vault's KV engine (`dlrsender`, `erlang`, `grafana`):

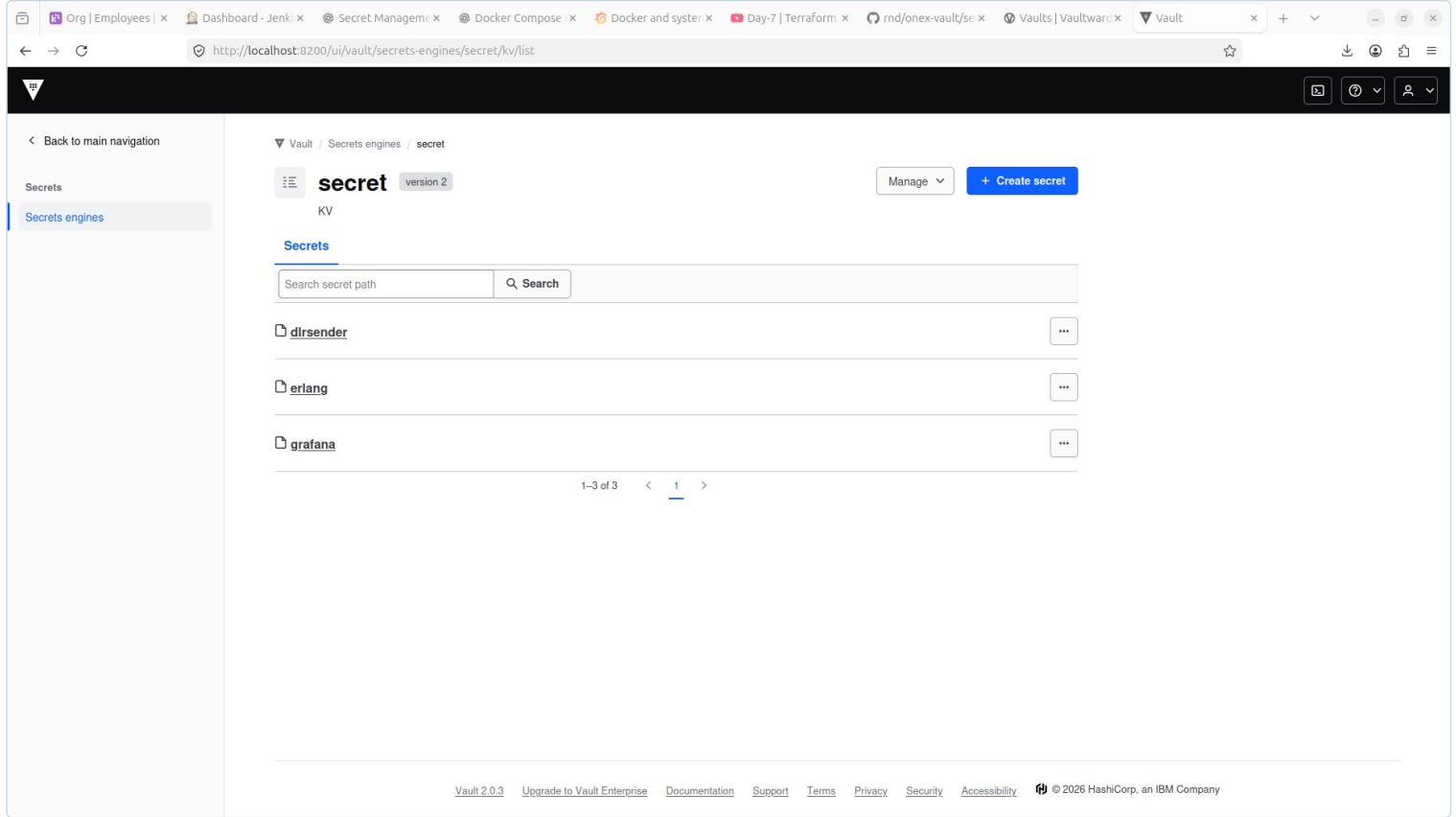


Figure 2 — Vault Web UI, KV v2 secrets engine listing: `dlrsender`, `erlang` and `grafana` secrets stored under the `secret/` mount.

6.2 OpenBao — curl / API Test

OpenBao was brought up the same way — as a container defined in the compose file (`docker-compose-sms-local-poc-openbao.yml`) — initialized, and logged into with the root token via `bao login` . As OpenBao is an API-compatible fork of Vault, the identical `curl` pattern (bearer/ `X-Vault-Token` header against `/v1/secret/data/<path>`) was used to retrieve the Grafana secret, confirming that existing Vault-style automation scripts port over to OpenBao without modification.

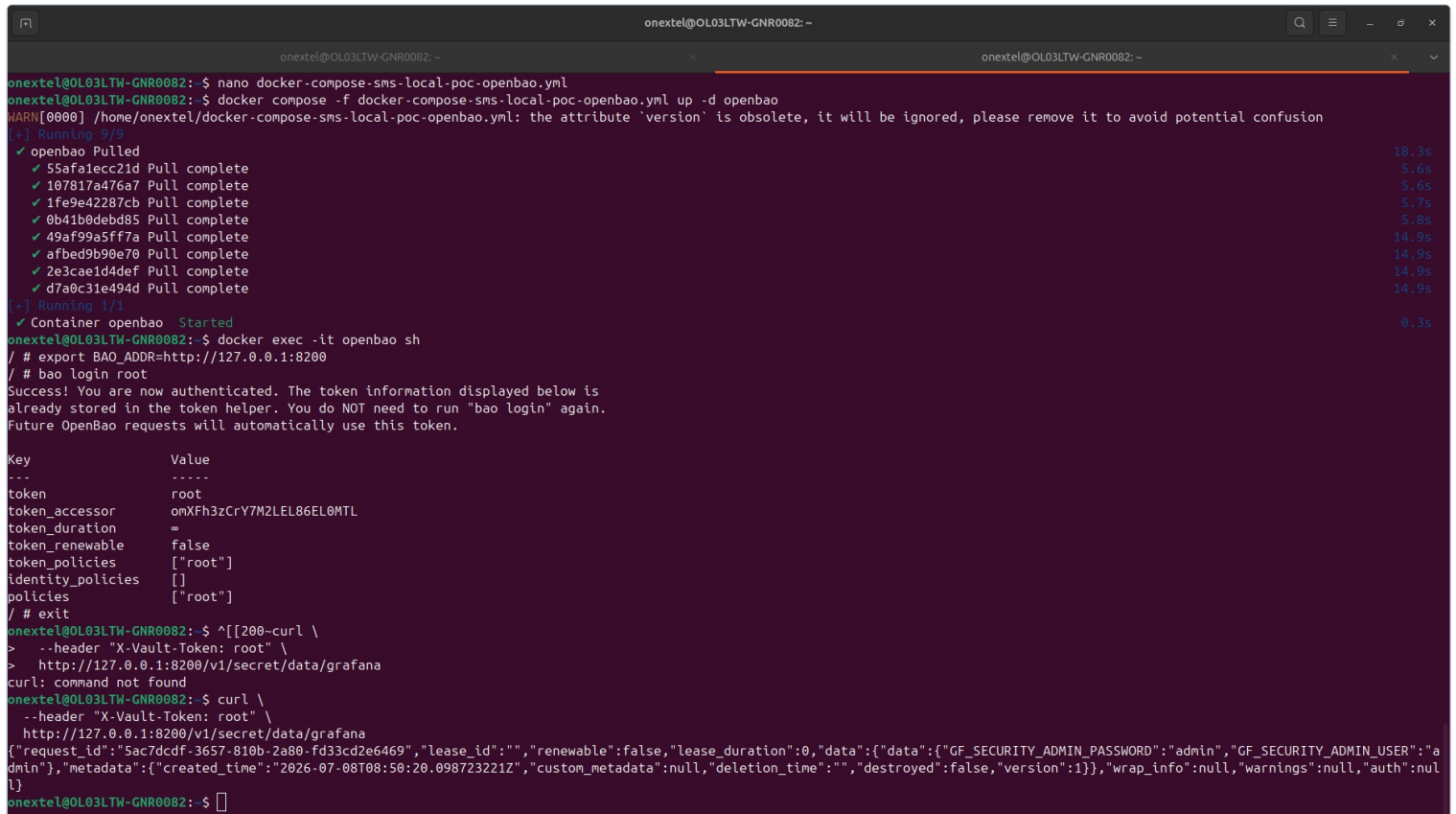


Figure 3 — Linux terminal: `docker compose up -d openbao` bringing up the OpenBao container, `docker exec -it openbao sh`, `bao login root`, and a `curl` call with an `X-Vault-Token` header against `http://127.0.0.1:8200/v1/secret/data/grafana`, returning the Grafana credentials as JSON — identical request shape to the Vault test.

Supporting evidence from OpenBao's own Web UI, showing the Grafana and Erlang secrets stored under the same KV path structure used for Vault:

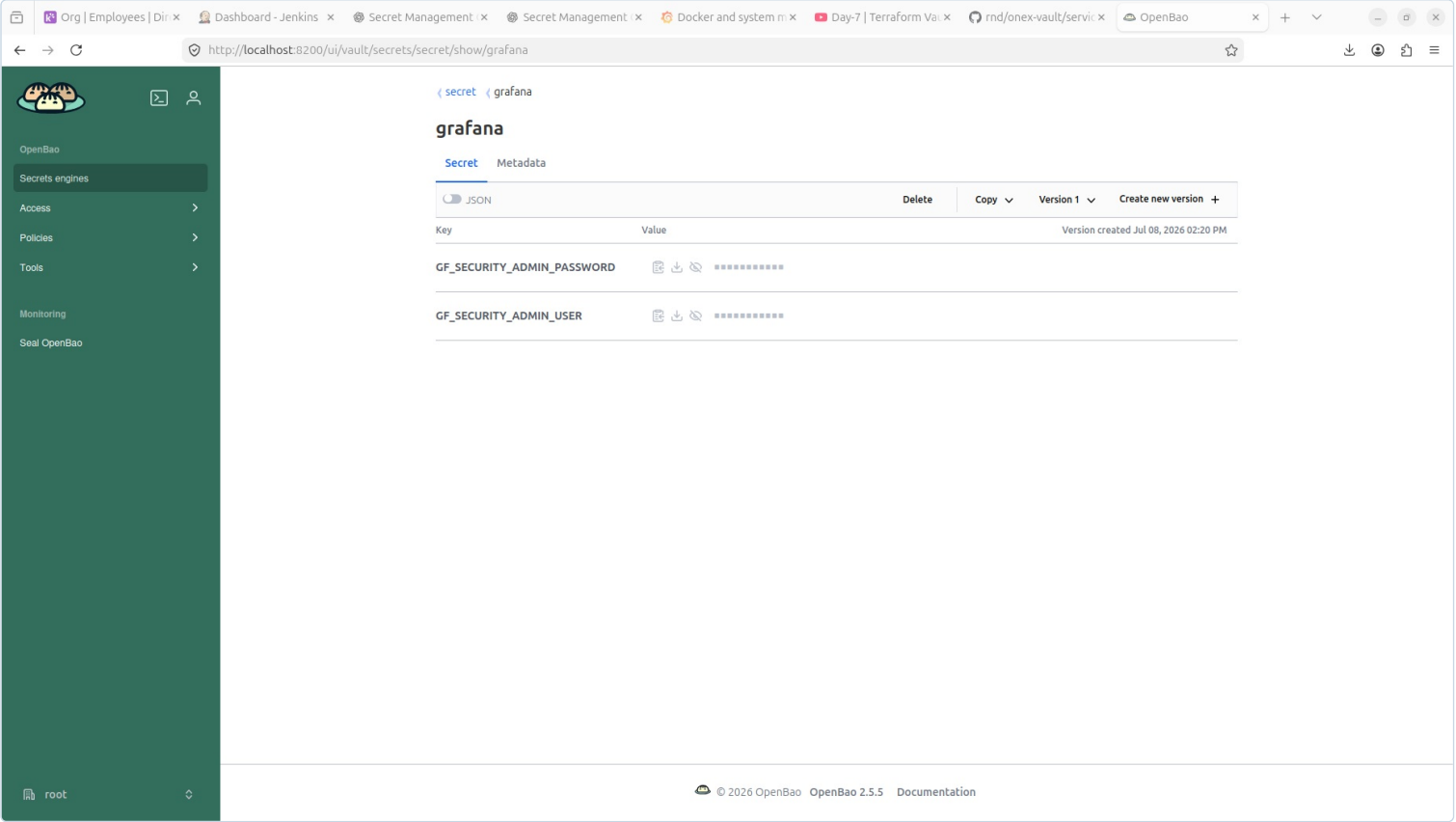


Figure 4 — OpenBao Web UI: grafana secret showing GF_SECURITY_ADMIN_USER and GF_SECURITY_ADMIN_PASSWORD key/value pairs.

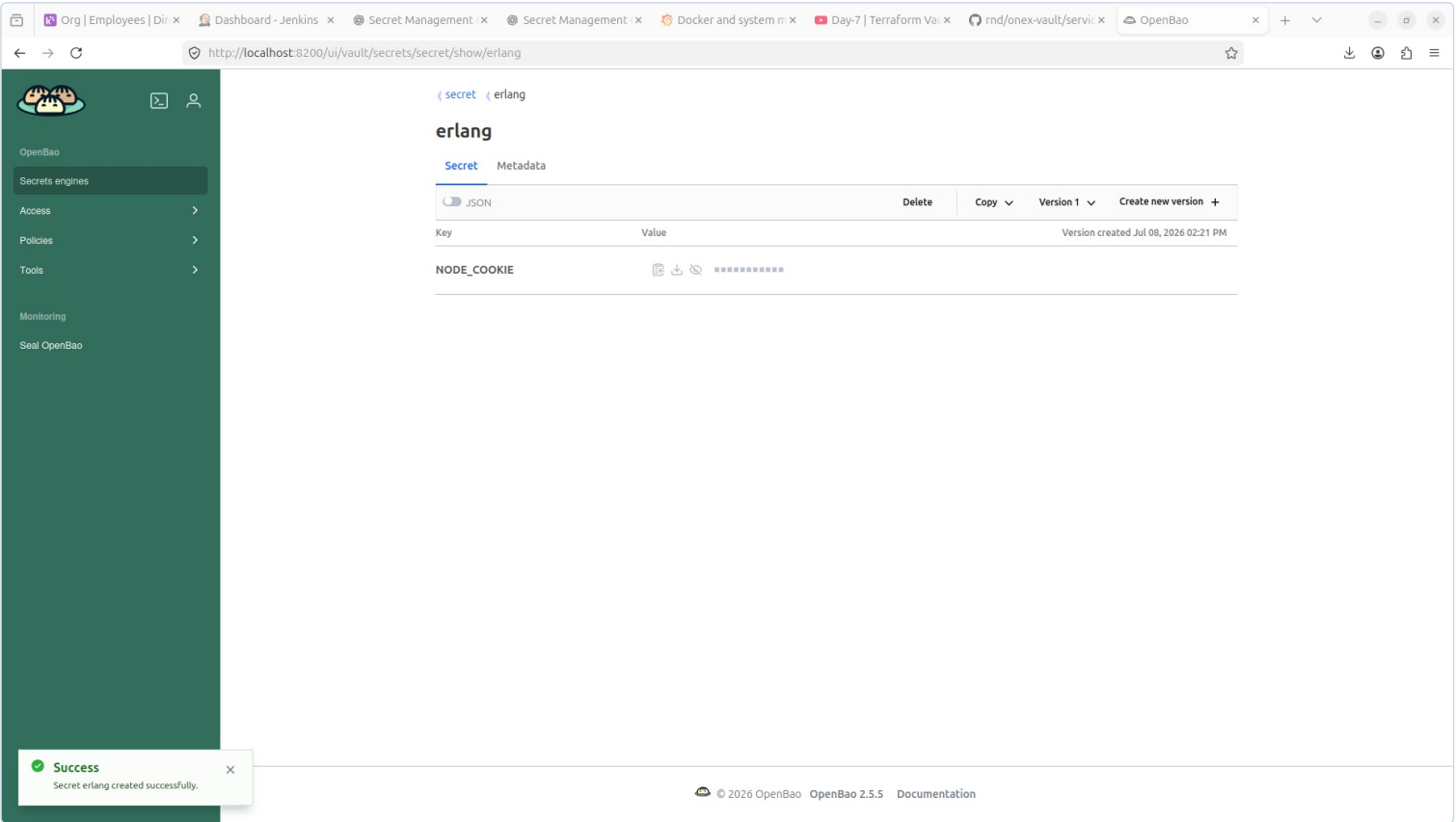


Figure 5 — OpenBao Web UI: erlang secret showing the NODE_COOKIE value used by the Erlang/RabbitMQ cluster.

6.3 Vaultwarden — API Limitations

Key finding: Vaultwarden cannot be tested with a simple, single curl request the way Vault/OpenBao can, and this is an architectural limitation rather than a configuration gap.

Vaultwarden is a self-hosted, Bitwarden-compatible **password manager**. It does not expose a simple REST endpoint that returns a secret value given only a static bearer token, the way HashiCorp Vault or OpenBao (or tools like Infisical) do. The official Bitwarden/Vaultwarden API surface is designed for **account and vault-sync management** consumed by the Bitwarden client applications, and retrieving an actual vault item requires walking through the full Bitwarden authentication flow — master password, client-side key derivation (KDF), and decryption of the encrypted vault blob on the client side — rather than issuing one authenticated HTTP call against a secret path.

In practice this means:

- There is no equivalent of `curl -H "X-Vault-Token: <token>" http://host:8200/v1/secret/data/grafana` for Vaultwarden.
- A machine wanting to read a secret would need to authenticate as a full Bitwarden user (email + master password + 2FA if enabled), unlock the vault client-side, and then use the Bitwarden/Vaultwarden CLI (`bw`) — not a single stateless API call.
- This makes Vaultwarden fundamentally oriented around *human* users logging into a client, not services or pipelines fetching a value programmatically.

As supporting evidence, the secrets that would need to be fetched (the Erlang cluster cookie, the Grafana admin credentials, and a sample "Kafka Brokers" secure note) were instead stored and can only be accessed through Vaultwarden's Web Vault UI shown below — there is no raw JSON secret-retrieval endpoint to screenshot, because none exists for this purpose:

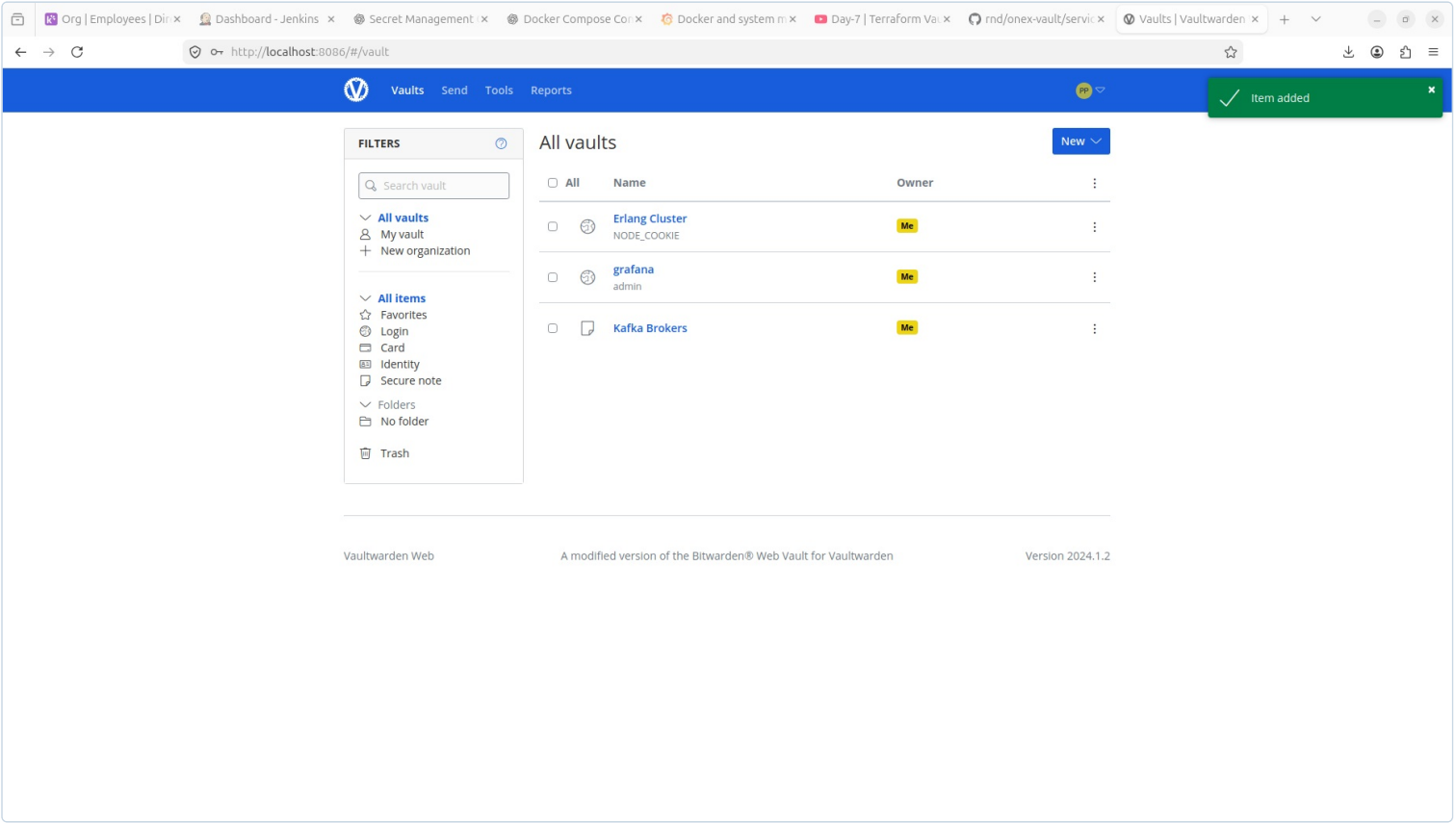


Figure 6 — Vaultwarden Web Vault UI (`http://localhost:8086/#/vault`): items stored as vault entries (**Erlang Cluster**, **grafana**, **Kafka Brokers**) — accessible only after full user login through the Bitwarden web/client authentication flow, not via a token-based curl call.

6.4 Phase 1 Summary

Solution	curl-based secret retrieval	Verdict
HashiCorp Vault	Confirmed working — single authenticated curl call returns JSON secret	PASS
OpenBao	Confirmed working — identical request shape to Vault, same result	PASS
Vaultwarden	Not possible with a single curl request; requires full client-side authentication/decryption flow	NOT SUITABLE FOR API-STYLE ACCESS

7. Phase 2 — CI/CD Integration Testing (Jenkins)

This phase validates whether secrets can be retrieved from within an actual Jenkins pipeline job — reflecting how the SMS department's build/deploy jobs would consume credentials in production, rather than a one-off terminal test.

7.1 HashiCorp Vault — Jenkins Pipeline

A Jenkins pipeline job (`poc-hcvault`) was created with a "Verify Vault" stage. The stage shell step ran `vault --version` , `vault status` , and `vault kv get secret/grafana` directly on the Jenkins agent against the running Vault container, and the console output confirms the stage completed successfully, returning the Grafana secret metadata and values (`GF_SECURITY_ADMIN_USER` / `GF_SECURITY_ADMIN_PASSWORD`) as part of the pipeline log.

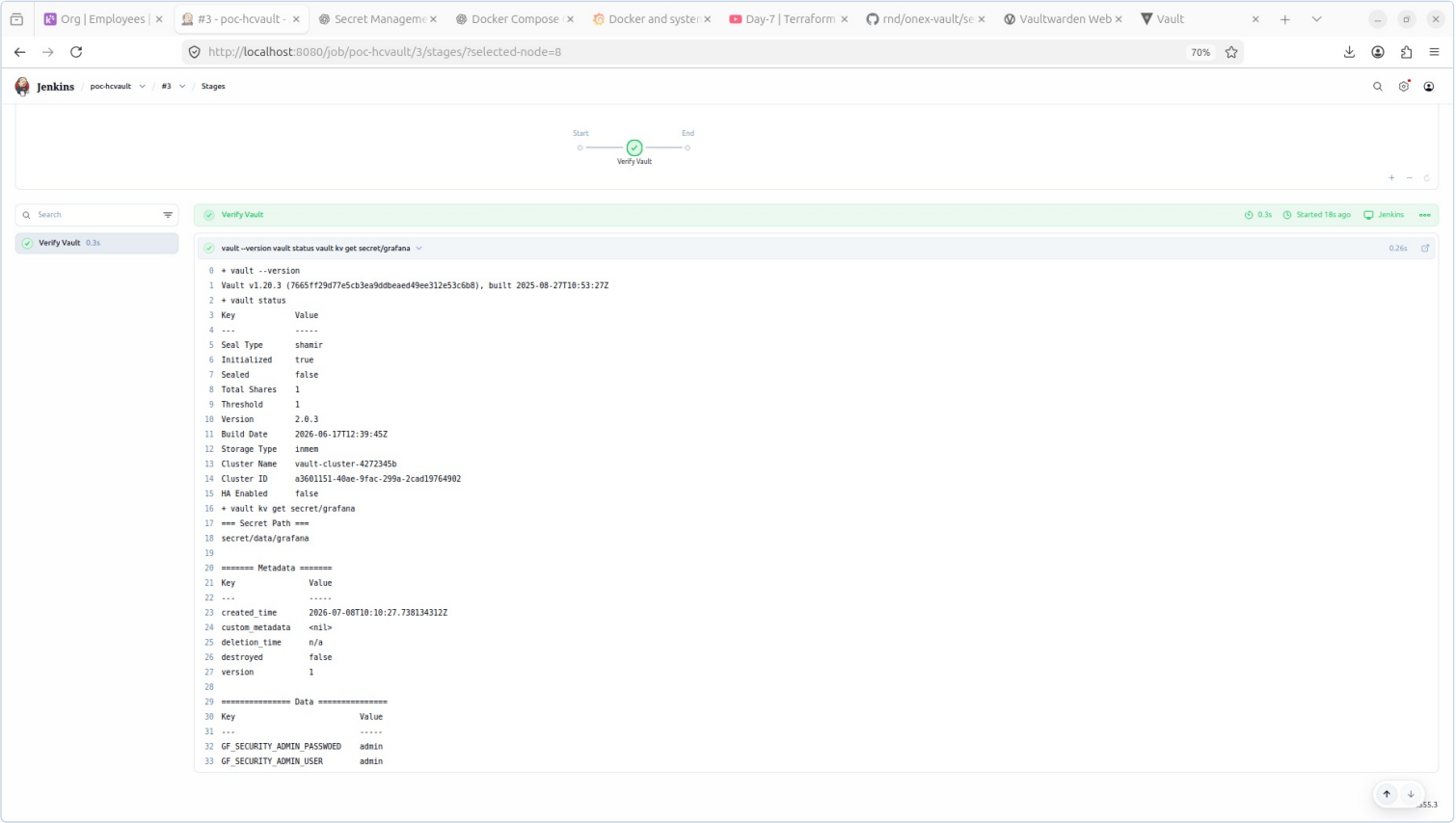


Figure 7 — Jenkins job `poc-hcvault` , build #3, "Verify Vault" stage: pipeline console log showing `vault --version` , `vault status` (unsealed, initialized), and `vault kv get secret/grafana` returning the Grafana admin credentials — executed as a normal pipeline stage, completed in 0.3s.

This demonstrates that Vault can be wired into a Jenkins stage using either the Vault CLI/API directly (as tested here) or, in a production rollout, the official Jenkins Vault plugin / Vault Agent for automatic token renewal and secret injection into environment variables.

7.2 OpenBao — Jenkins Pipeline

The equivalent pipeline job (`openbao-poc`) was created with a "Verify OpenBao" stage. The stage logged into OpenBao (`bao login root`) and ran `bao kv get` against both the `grafana` and `erlang` secret paths, with the console output confirming both secrets were retrieved successfully inside the Jenkins job.

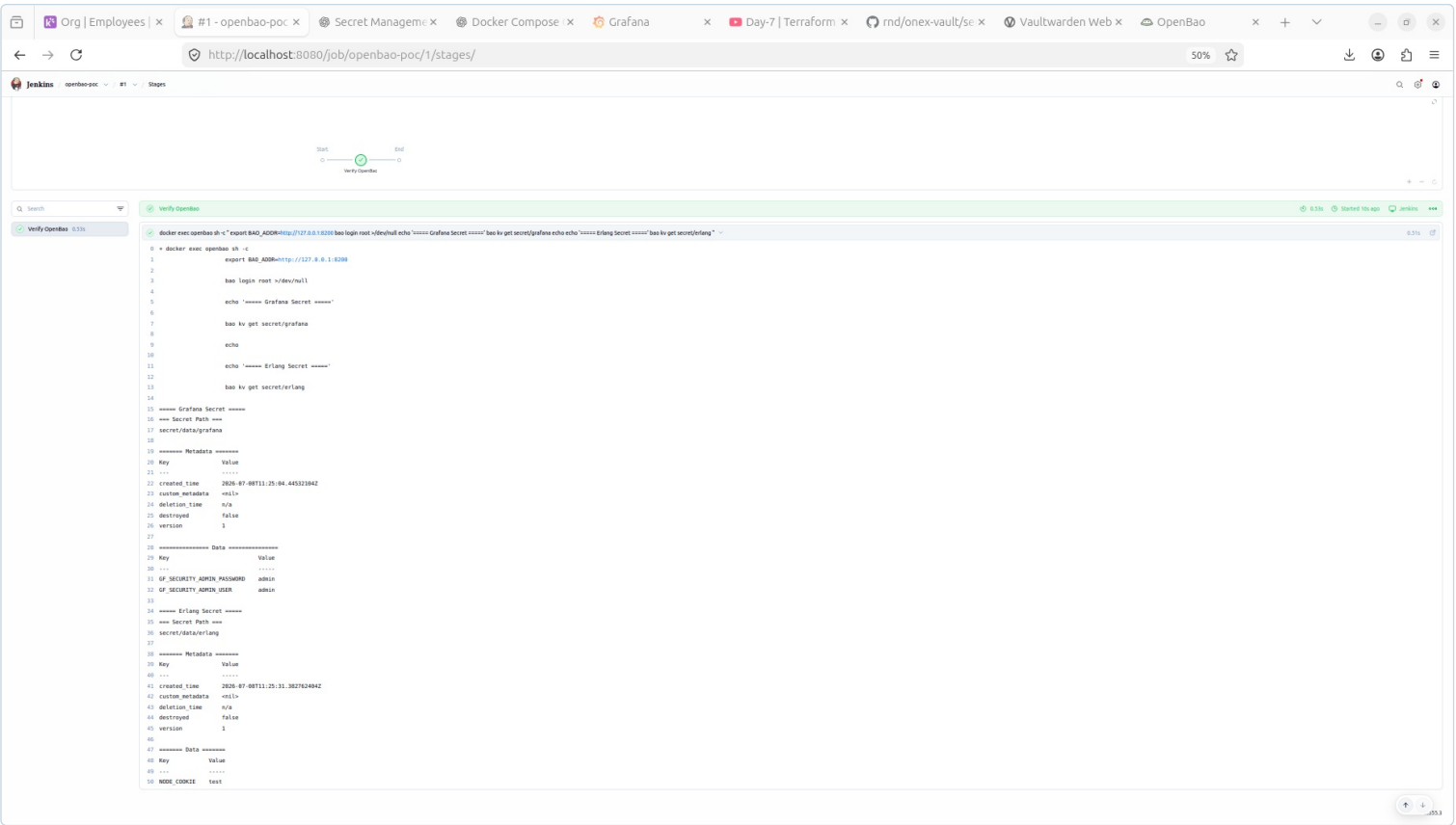


Figure 8 — Jenkins job `openbao-poc` , build #1, "Verify OpenBao" stage: pipeline console log running `docker exec openbao sh -c "bao login root && bao kv get secret/grafana && bao kv get secret/erlang"` , successfully printing both the Grafana credentials and the Erlang `NODE_COOKIE` .

Because OpenBao is API/CLI-compatible with Vault, this pipeline stage required no different tooling — the same shell pattern used for Vault worked unchanged, simply pointed at the OpenBao container.

7.3 Vaultwarden — CI/CD Suitability

Vaultwarden is a password management solution compatible with the Bitwarden ecosystem. While secrets can technically be accessed using the Bitwarden CLI after user authentication, Vaultwarden is not designed to act as a backend for CI/CD systems such as Jenkins. Unlike HashiCorp Vault and OpenBao, it lacks native mechanisms for automated infrastructure secret injection, dynamic secrets, and fine-grained machine authentication. Therefore, Vaultwarden is more suitable for managing credentials for human users than for DevOps automation.

No Jenkins pipeline stage was built for Vaultwarden in this POC, since doing so would require embedding a human user's master password (and handling any 2FA) inside a Jenkins credential/stage — which defeats the purpose of machine-to-machine secret automation and would itself introduce a new secret-handling risk into the pipeline. This is the same architectural limitation identified in Phase 1, and it applies equally, if not more, to a CI/CD context.

7.4 Phase 2 Summary

Solution	Jenkins pipeline integration	Verdict
HashiCorp Vault	Confirmed — pipeline stage retrieved Grafana secret via CLI/API	PASS
OpenBao	Confirmed — pipeline stage retrieved Grafana & Erlang secrets via CLI/API	PASS
Vaultwarden	Not attempted — no supported machine-authentication path for automated pipelines	NOT SUITABLE FOR CI/CD

8. Evaluation Against Requested Parameters

Each parameter requested for this evaluation is addressed below, based on the POC and supporting desk research.

8.1 Installation and setup complexity

Vault and OpenBao share essentially the same setup workflow: pull the image, start the container, run `vault / bao operator init`, unseal, then log in with the resulting root token — as carried out in this POC. Vaultwarden required only a single container with minimal environment configuration to be usable, making it the fastest to stand up, but that simplicity is a direct reflection of its narrower feature set.

8.2 Secret storage capabilities

Vault/OpenBao store arbitrary key-value secrets (as used here for Grafana credentials and the Erlang cookie) and can additionally generate short-lived, dynamic credentials for databases, cloud providers, and PKI/SSH — none of which Vaultwarden supports, since it is built around discrete "vault items" (logins, secure notes, cards) intended for individual/team password management.

8.3 Access control and authentication mechanisms

Vault/OpenBao support fine-grained HCL policies plus multiple auth methods (tokens, AppRole for machines, LDAP/OIDC for humans, Kubernetes service-account auth) — all evaluated as applicable to both human operators and automated systems. Vaultwarden's access model is built around Bitwarden user accounts, organizations, and collections — well suited to sharing a password with a colleague, but with no machine-oriented authentication method (no AppRole/service-account equivalent).

8.4 Audit logging and security features

Vault and OpenBao both support built-in audit devices that log every request and response (including who accessed which secret and when), which is important for compliance in a production credential store. Vaultwarden logs vault activity for the web UI but does not provide an equivalent dedicated audit-device/API for automated log shipping.

8.5 API and automation support

As demonstrated directly in Phase 1, Vault and OpenBao expose a full REST API that a single authenticated `curl` call (or any HTTP client/SDK) can use to fetch a secret. Vaultwarden's API is oriented around account/vault synchronization for its own client apps and does not offer an equivalent single-call secret-retrieval endpoint.

8.6 Integration with CI/CD tools (Jenkins, GitLab, etc.)

Confirmed directly in Phase 2 for Vault and OpenBao via working Jenkins pipeline stages; both also have official/community Jenkins and GitLab CI plugins available for production use (Vault Agent, GitLab's native Vault integration). Vaultwarden has no equivalent CI/CD plugin ecosystem, consistent with the finding in Section 7.3.

8.7 Kubernetes and container platform support

Vault has a mature, official Helm chart, Vault Agent Injector (automatic sidecar secret injection), and a Secrets Store CSI driver. OpenBao, being a compatible fork, offers the same integration patterns through its own community-maintained Helm chart and CSI provider, though the surrounding ecosystem is younger. Vaultwarden runs comfortably as a container/Helm-deployable app itself, but has no equivalent tooling to inject secrets into other pods.

8.8 High availability and backup options

Vault and OpenBao both support HA clustering using Integrated Storage (Raft) with leader election and failover, plus a documented snapshot/restore process for backups — this POC ran a single (non-HA) node for simplicity, consistent with a local proof-of-concept, but the HA path is well documented for both. Vaultwarden is designed to run as a single instance; "backup" is effectively a copy of its database file and attachments, with no built-in clustering/failover.

8.9 Ease of administration

Vaultwarden's Bitwarden-style Web UI is the simplest to administer day-to-day for end users managing their own passwords. Vault and OpenBao share the same administration surface (Web UI, CLI, API) which is more powerful — policies, auth methods, secret engines, audit devices — but requires more upfront learning for an administrator.

8.10 Licensing and cost considerations

OpenBao is fully open source (MPL-2.0) with no usage restrictions, governed by the Linux Foundation. HashiCorp Vault's Community Edition is source-available under the Business Source License (BSL-1.1), which carries usage restrictions for competing/hosted offerings but remains free to self-host for internal use as done in this POC; Vault Enterprise is a paid product. Vaultwarden is open source (AGPL-3.0)

and free to self-host.

8.11 Community support and documentation

HashiCorp Vault has the largest and most mature community and documentation set, given its long-standing position as the de-facto standard. OpenBao's documentation closely mirrors Vault's (by design, given the shared codebase) and its community is growing quickly post-fork with Linux Foundation backing. Vaultwarden has an active, well-documented community specifically around Bitwarden-compatible password management use cases.

9. Pros and Cons per Solution

9.1 HashiCorp Vault

Pros

- Industry-standard, mature feature set (dynamic secrets, PKI, transit encryption)
- Confirmed working curl-based API access and Jenkins pipeline integration
- Rich policy engine and wide range of auth methods
- Official Helm chart, Agent Injector, huge ecosystem/documentation

Cons

- BUSL license carries usage restrictions compared to a fully open license
- Steeper learning curve for full policy/auth configuration
- Enterprise features (advanced HA, DR replication) require a paid license

9.2 OpenBao

Pros

- Fully open source (MPL-2.0), no usage restrictions, Linux Foundation governed
- API/CLI-compatible with Vault — confirmed identical curl and Jenkins pipeline behaviour in this POC
- Same policy, auth-method, and secrets-engine model as Vault
- Future-proof against further Vault license changes

Cons

- Younger project — smaller ecosystem, fewer third-party integrations than Vault today
- Enterprise-grade support offerings are still maturing

9.3 Vaultwarden

Pros

- Extremely simple to install and administer
- Excellent for human-facing password/credential sharing (teams, personal use)
- Fully open source (AGPL-3.0), very lightweight resource footprint

Cons

- No single-call, token-based API for automated secret retrieval (confirmed in Phase 1)
- No native CI/CD integration or machine-authentication method (confirmed in Phase 2)
- No dynamic secrets, no audit devices, no HA/clustering
- Not designed for infrastructure/DevOps automation use cases

10. Scorecard

A simple 1–5 scoring (5 = best) across the requested evaluation parameters, based on the findings above:

Parameter	HashiCorp Vault	OpenBao	Vaultwarden
Installation & setup	3	3	5
Secret storage capability	5	5	2
Access control / auth	5	5	2
Audit logging	5	5	2
API / automation support	5	5	1
CI/CD integration	5	5	1
Kubernetes / container support	5	4	2
HA & backup	5	5	2
Ease of administration	3	3	5
Licensing & cost	3	5	5
Community & documentation	5	4	4
Total (out of 55)	49	49	31

11. Recommendation / Final Verdict

Both Phase 1 and Phase 2 of this POC confirmed that **HashiCorp Vault and OpenBao** are capable, production-viable secret managers for this environment: both returned the Grafana and Erlang secrets via a single authenticated `curl` call, and both were successfully wired into a real Jenkins pipeline stage using the same shell/CLI pattern. **Vaultwarden**, while very easy to run and administer, was confirmed in both phases to be architecturally unsuited to API-driven or CI/CD-driven secret retrieval, since it requires a full human login/decryption flow rather than a stateless token-based API call.

Final Verdict: OpenBao

For the SMS department's requirement — securing the Grafana admin credentials, the `dlrsender` service secrets, and the Erlang cluster cookie that are currently sitting in plaintext inside the compose stack — **OpenBao** is the recommended solution, for the following reasons:

- It delivered *identical* functional results to HashiCorp Vault in every test performed in this POC (API retrieval and Jenkins pipeline integration), so there is no functional trade-off versus Vault.
- It is **fully open source (MPL-2.0)** with no usage restrictions, removing the licensing ambiguity introduced by Vault Community Edition's BUSL-1.1 terms — an important consideration for a long-term internal platform component.
- It is governed by the Linux Foundation and remains command/API-compatible with Vault, so the team retains the option to adopt Vault Enterprise later, or continue on OpenBao, without needing to re-architect secret paths, policies, or pipeline integrations.
- It directly addresses the security concern that triggered this exercise: the Grafana, `dlrsender`, and Erlang secrets can be moved out of the compose file/plaintext env vars and into OpenBao's access-controlled, audit-logged KV store, then injected into containers and Jenkins jobs via the API — exactly as demonstrated in this POC.

Vaultwarden is not recommended as the primary infrastructure secret store, but may still have a place alongside OpenBao purely as a password manager for human operators (e.g. sharing personal/admin login credentials among team members), which is the use case it is actually designed for.

12. Conclusion

This POC set out to compare HashiCorp Vault, OpenBao, and Vaultwarden from a DevOps and infrastructure perspective, using the department's existing SMS Docker Compose stack as the basis for testing, and to determine which tool should replace the plaintext secrets (Grafana credentials, `dlrsender` secrets, and the Erlang cluster cookie) currently embedded in that stack.

Across installation, secret storage, access control, audit logging, API/automation support, CI/CD integration, Kubernetes support, HA/backup, administration effort, licensing, and community support, HashiCorp Vault and OpenBao performed comparably well and both were proven, hands-on, to support curl-based API retrieval and Jenkins pipeline integration. Vaultwarden performed strongly on ease of use but was confirmed unsuitable for automated, machine-driven secret retrieval in both the API test (Phase 1) and the CI/CD test (Phase 2).

Based on the combined desk research and hands-on POC results, **OpenBao** is recommended as the secret management solution to adopt for securing the SMS department's infrastructure secrets going forward, with Vaultwarden retained only as an optional, complementary tool for human-facing password sharing.

End of report. All screenshots in this document were captured directly from the POC environment described in Section 1.